Functional Specification Document (FSD)

TSI Ledger Service: Multi-Tenant Double-Entry Engine with Cryptographic Tamper-Evidence

1. Executive Summary & Core Guarantees

The TSI Ledger Service is designed as a centralized, highly reliable, and isolated double-entry accounting engine accessible strictly via API. The system serves external applications ("Apps"), allowing them to manage independent charts of accounts and post transactions securely.

To comply with institutional standards and frameworks such as the **Bank Secrecy Act (BSA)**, the ledger incorporates strict data validation alongside sequential cryptographic hashchaining to provide immutable proof against financial data tampering.

Core Architectural Invariants

- **Double-Entry Invariant:** Every transaction must possess at least two balancing legs. The arithmetic sum of all debits must perfectly match the sum of all credits.

$$\sum Debits = \sum Credits$$

- **Strict Immutability:** Once a journal entry is committed, it can never be updated, deleted, or reordered. Financial corrections must occur via explicit reversal entries.
- **Tenant Isolation:** Data remains logically isolated per App. An API request authenticated for a specific App cannot access or modify data belonging to another.
- **Tamper Detection:** Historical entry validation relies on a block-to-block cryptographic linkage. Any out-of-band manipulation breaks the chain deterministically.

2. System Architecture & Data Model

The system utilizes a clean relational architecture optimized for fast lookups and exact numeric computations. Floating-point types are strictly prohibited; all monetary limits are stored using arbitrary-precision decimals.

Account Classifications & Natural Balances

Account Type	Natural Balance	Increase Effect	Decrease Effect
Asset	Debit	Debit	Credit
Liability	Credit	Credit	Debit
Equity	Credit	Credit	Debit
Revenue	Credit	Credit	Debit
Expense	Debit	Debit	Credit

Entities Structure

- **App:** The primary tenant wrapper containing a unique ID, application name, and hashed API keys.
- **Account:** Individual ledger lines representing distinct tracking units (e.g., Cash, Accounts Receivable). Identified by a unique system ID and an app-scoped user-defined `code`.
- **Journal Entry:** The structural anchor of a transaction containing audit timestamps, descriptions, external references, and cryptographic block hashes.
- **Entry Line:** Individual debit/credit records linked back to a parent Journal Entry and a targeted Account.

3. Authentication & Tenant Configuration

Every incoming HTTP request must include the `X-API-Key` header. The service uses a two-tier API key structure (live and test) mapped directly to an active `App` context.

> **Security Operational Rule**
>
> The underlying system evaluates the API key against a secure hash index at the database gateway level. If no matching app context is resolved, the system terminates the request immediately with an HTTP `401 Unauthorized` block.

4. Core Ledger Engine & Entry Posting

Endpoint: `POST /v1/entries`

Allows authenticated systems to log multi-line financial transactions. The engine processes inputs transactionally using ACID guarantees.

Strict Validation Workflow

1. Confirm all specified `account_code` parameters are registered and active within the tenant's context.
2. Validate the mathematical balance: total debits must perfectly match total credits down to the smallest subunit (e.g., 0.01).
3. Enforce request idempotency using the mandatory `Idempotency-Key` header to prevent double-posting due to network retries.

Sample Payload Format

```
{
  "reference_id": "TXN-2026-8891",
  "description": "Enterprise Subscription Settlement - Invoice #451",
  "posted_at": "2026-07-06T06:12:00Z",
  "lines": [
    {
      "account_code": "1010",
      "type": "DEBIT",
      "amount": 1250.00
    },
    {
      "account_code": "4000",
      "type": "CREDIT",
      "amount": 1250.00
    }
  ]
}
```

5. Cryptographic Hashchaining (BSA Compliance Proof)

To establish non-repudiation and verify compliance with internal and external federal frameworks (e.g., Bank Secrecy Act records integrity), each Journal Entry calculates a sequential cryptographic link.

Mathematical Hashing Chain Logic

Every transaction is assigned a gapless, strictly incrementing sequential integer (n) scoped entirely within that specific App context. The structural block hash (H_n) is computed deterministically:

$$P_n = SHA256(CanonicalJSON(Metadata_n + Lines_n))$$

$$H_n = SHA256(P_n + H_{n-1})$$

Where P_n represents the payload hash, and H_{n-1} represents the previous transaction block's overall hash. For the initial transaction ($n = 1$), the system substitutes a standardized, hardcoded 64-character zero string as the genesis pointer.

Endpoint: `GET /v1/audit/verify`

Iterates over the hash values sequentially to ensure structural consistency. If a single historical row's data is modified out-of-band, the validation sequence will fail instantly, exposing the entry sequence index of the breach.

6. Financial Reporting APIs

6.1 Trial Balance (`GET /v1/reports/trial-balance`)

Aggregates absolute debit and credit totals across all accounts up to the specified target date context parameter (`as_of_date`). Used primarily by automated validation scripts to test structural balance health.

6.2 Balance Sheet (`GET /v1/reports/balance-sheet`)

Compiles account groupings to dynamically evaluate and output the classic fundamental accounting equation:

$$Assets = Liabilities + Equity$$

Asset account balances are computed as $\sum Debits - \sum Credits$, whereas Liabilities and Equity balances track naturally as $\sum Credits - \sum Debits$.